E-commerce Recommendation Engine — Day 1 Summary

Parvaaz project · 22 Oct 2026 · Status: **Planning complete → Build phase started** · All details live in the project folder (CLAUDE.md is the single source of truth)

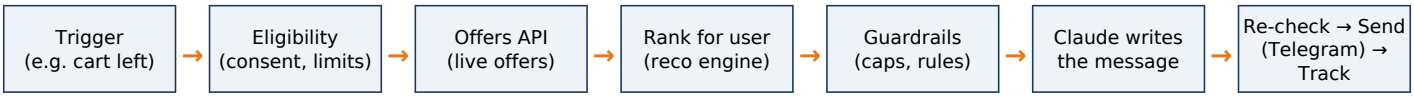
1. What we are building

An AI-powered recommendation platform for an Indian online store: it **recommends products** each shopper is likely to want, **creates personalised offers** at the right moment, **sends them on Telegram** with a message written by Claude AI, and lets users **chat with a shopping assistant** — all protected by **guardrails** so the AI never invents products, prices or discounts and never spams.

2. Key decisions taken today

Area	Decision
Approach	Industry-standard 3-stage recommender: **find candidates** (popularity, "bought together", ALS, content similarity) → **rank** (LightGBM) → **apply rules** (stock, budget, variety).
Data	We **generate our own realistic data** (10k users, 3k products, ~1M actions, Apr–Sep 2026) with hidden shopper tastes planted in it, so we can prove the models really learn. Parameters: docs/DATA_PARAMETERS.md.
Standard format	One fixed data format for the whole system. Real datasets (Amazon, Myntra, RetailRocket, any CSV) plug in through a small mapping file; missing data switches features off instead of crashing.
What makes us different	No "fridge after fridge" (accessories after a purchase) · every recommendation shows **why** · user controls (budget, gift mode, "not interested") · budget-aware for Tier-2/3 India.
AI + guardrails	**"AI proposes, rules decide."** Prices, stock, product IDs and offer values always come from our database. Checks on input, tools, output; safe fallback if the AI fails; red-team test of 50+ tricky prompts.
Offers	Created by **our own Offer Engine**, served by our own Offers API; the recommendation engine picks the best match; Claude **only writes the wording**.
Channel & AI	Telegram bot (user opts in with /start, out with /stop) · Claude API (fast model for messages, smarter model for chat).
Security	All keys kept in a private .env file — never in code, git, chats or AI prompts.

3. Approved offer flow



If the AI fails → a plain template message is sent. If any rule fails → nothing is sent and the reason is logged.

4. How we build it — phases in parallel chats

Wave	Phase (prompt file in phases/)	What it produces
1 · first, alone	0 Foundation	Folders, shared settings & names, sample data, placeholder modules, health-check script
2 · in parallel	1 Data generator + importers	Full synthetic dataset; converters for Amazon/Myntra/other real data
	2 Recommendation engine	Algorithms, ranking, rules, reasons, evaluation scores
	3 Offer engine + Offers API	Offer creation, offer guardrails, approval for big discounts
	4 AI layer + guardrails	Claude message writer, chat assistant, safety checks, red-team test
	5 Telegram bot	Account linking, commands, buttons, rate-limited sending
	6 API + demo website	Backend endpoints + Streamlit storefront with user controls
3 · last	7 Integration	Everything connected, end-to-end tests, final evaluation report

Each phase chat edits only its own folder and uses placeholder versions of the others, so nobody waits. Suggested timeline ≈ 8 weeks (week 1 foundation · weeks 2–5 parallel build · 6–7 integration · 8 report).

5. Files created in the project folder

File	Purpose
CLAUDE.md	Single source of truth: decisions, data format, function names, rules for every chat
phases/ (README, PHASE_0 … PHASE_7, PROGRESS, NOTES)	Ready-to-paste prompt for each phase, run order, progress tracker
docs/BEGINNER_GUIDE.md	Project explained for zero-knowledge readers + glossary + 8-week study plan
docs/SETUP_GUIDE.md	Step-by-step setup: Python, .env, Claude key, Telegram bot, troubleshooting
docs/DATA_PARAMETERS.md · docs/DATA_STANDARD.md	How synthetic data is generated · standard format for real datasets

6. Next steps

① Fix team size & timeline ② Everyone completes docs/SETUP_GUIDE.md ③ Get Claude API key + create Telegram bot (keys go into .env only) ④ Run Phase 0 in a new chat ⑤ Start Phases 1–6 in parallel chats